

Relatório de Execução - Challenge 2 - Serverest API

Autor: Gustavo Santos Queiroz

Ferramentas: Postman, Confluence.

Ambiente de execução: Windows 11, Postman WEB no Navegador Brave.

LINK POSTMAN DA COLLECTION, COM ENVIRONMENT INCLUÍDO: <https://gus888-7291.postman.co/workspace/Gus-Workspace~db9dc619-a29f-4814-8638-f09253748b8e/collection/51860449-bf66ebe8-d631-4325-8fb4-55c18fd60df8?action=share&creator=51860449&active-environment=51860449-e1a45886-dec3-41ce-8cd0-5f08e3fbfd59>

COLLECTION EM .JSON ENVIADA SEPARADAMENTE

1. Execução dos Testes: Tour do Supermercado

CT01: Cadastrar Usuário Administrador

- **Endpoint:** POST `{{url}}/usuarios`

- **Script (Pre-request):**

```
1 pm.environment.set("email_tour", "cliente" + Date.now() + "@tour.com");
2 // cria uma variável (.set) no ambiente (.environment) com o nome "email_tour" e o valor "cl
```

- **Request Body (JSON):**

```
1 {
2   "nome": "Gustavo Tour",
3   "email": "{{email_tour}}",
4   "password": "teste",
5   "administrador": "true"
6 }
```

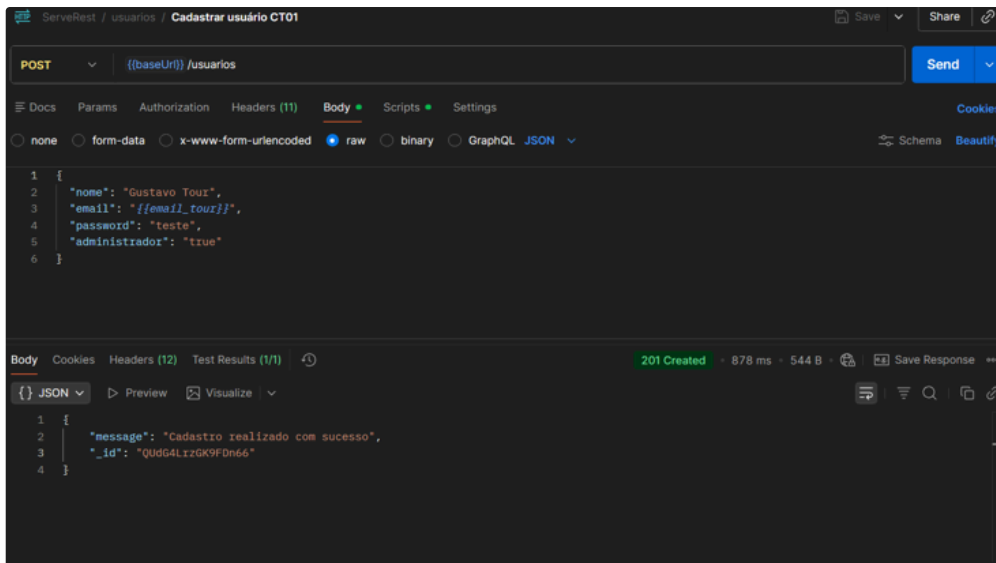
- **Response Body (JSON):**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "QUdG4LrzGK9FDn66"
4 }
```

- **Script de Validação (Post-request):**

```
1 pm.test("Status deve ser 201 Created", function () {
2   pm.response.to.have.status(201);
3 });
```

- **Evidência:**



CT02: Realizar Login

- **Endpoint:** POST `{{url}}/login`
- **Request Body (JSON):**

```
1 {
2   "email": "{{email_tour}}",
3   "password": "teste"
4 }
```

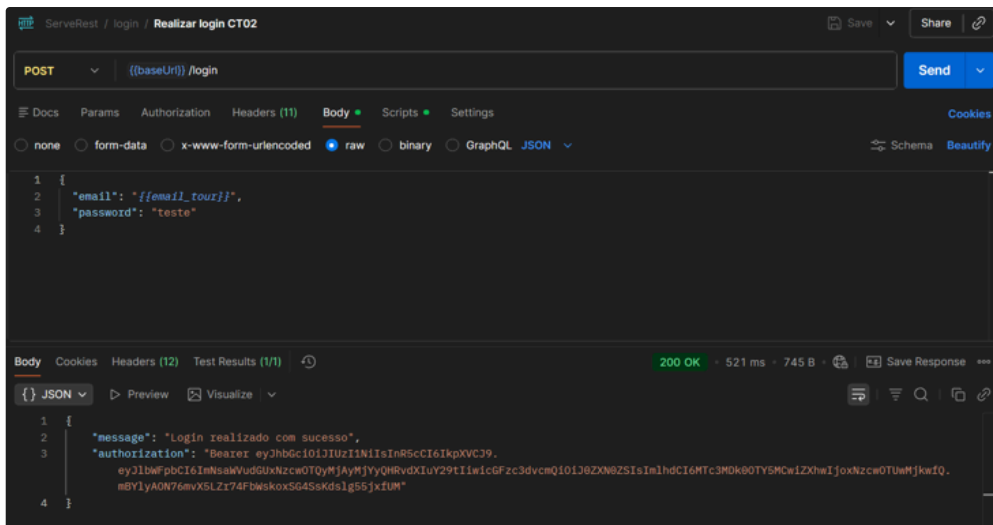
- **Response Body (JSON):**

```
1 {
2   "message": "Login realizado com sucesso",
3   "authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bWVpbiCI6ImNsaWVudGUxNzcwOTU",
4 }
```

- **Script (Post-request):**

```
1 pm.test("Status deve ser 200 OK", function () {
2     pm.response.to.have.status(200);
3 });
4
5 var resposta = pm.response.json();
6     // cria uma variável chamada resposta e armazena o JSON da resposta
7 pm.environment.set("token_tour", resposta.authorization);
8     // armazena o token (.authorization) na variável de ambiente token_tour
```

- **Evidência:**



CT03: Cadastrar Produto no Estoque

- **Endpoint:** POST {{url}}/produtos
- **Authorization:** {{token_tour}}
- **Request Body (JSON):**

```
1 {
2   "nome": "Produto Tour 2026",
3   "preco": 100,
4   "descricao": "Teste Automatizado",
5   "quantidade": 50
6 }
```

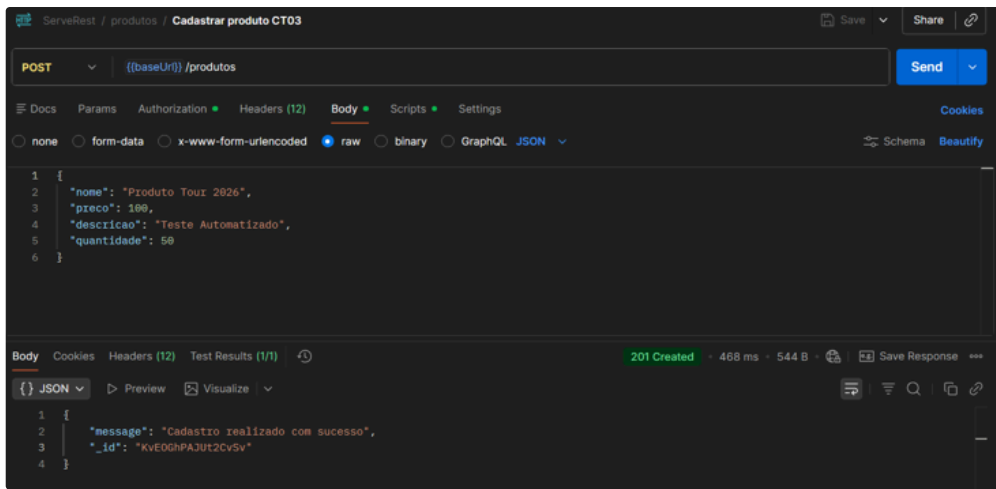
- **Response Body (JSON):**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "KvEOGhPAJU2CvSv"
4 }
```

- **Script (Post-request):**

```
1 pm.test("Produto criado com sucesso (201)", function () {
2   pm.response.to.have.status(201);
3 });
4
5 var resposta = pm.response.json();
6 pm.environment.set("id_produto_tour", resposta._id);
7 // armazena o id do produto criado na variável de ambiente {{id_produto_tour}}
```

- **Evidência:**



CT04: Listar Produto via ID

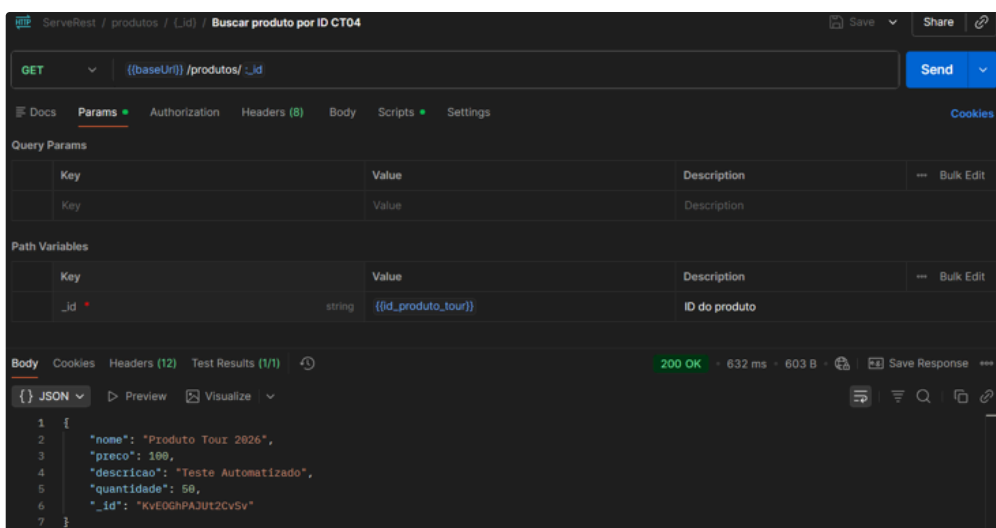
- Endpoint: `{{baseUrl}}/produtos/:id`
- `_id`: `{{id_produto_tour}}`
- Response Body (JSON):

```
1 {  
2   "nome": "Produto Tour 2026",  
3   "preco": 100,  
4   "descricao": "Teste Automatizado",  
5   "quantidade": 50,  
6   "_id": "KvEOGhPAJU2CvSv"  
7 }
```

- Script (Post-request):

```
1 pm.test("Lista de produtos carregada (200)", function () {  
2   pm.response.to.have.status(200);  
3 });
```

- Evidência:



CT05: Adicionar Item ao Carrinho

- Endpoint: `POST {{url}}/carrinhos`

- **Authorization:** `{{token_tour}}`

- **Request Body (JSON):**

```
1 {
2   "produtos": [
3     {
4       "idProduto": "{{id_produto_tour}}",
5       "quantidade": 1
6     }
7   ]
8 }
```

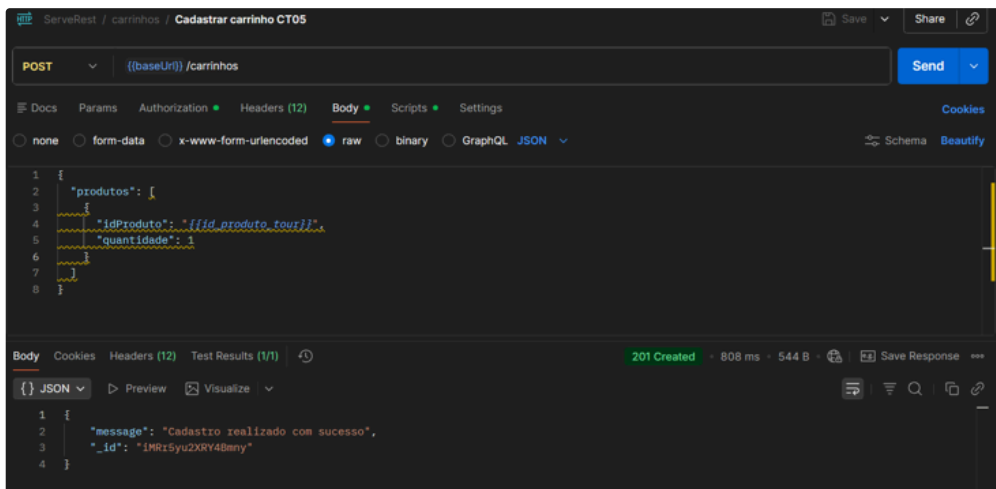
- **Response Body (JSON):**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "qbMqntef4iT0wWfg"
4 }
```

- **Script (Post-request):**

```
1 pm.test("Item adicionado ao carrinho (201)", function () {
2   pm.response.to.have.status(201);
3 });
```

- **Evidência:**



CT06: Concluir Compra (Checkout)

- **Endpoint:** `DELETE {{url}}/carrinhos/concluir-compra`

- **Authorization:** `{{token_tour}}`

- **Request Body (JSON):** (Vazio)

- **Response Body (JSON):**

```
1 {
2   "message": "Registro excluído com sucesso",
3   "_id": "qbMqntef4iT0wWfg"
4 }
```

- **Script (Post-request):**

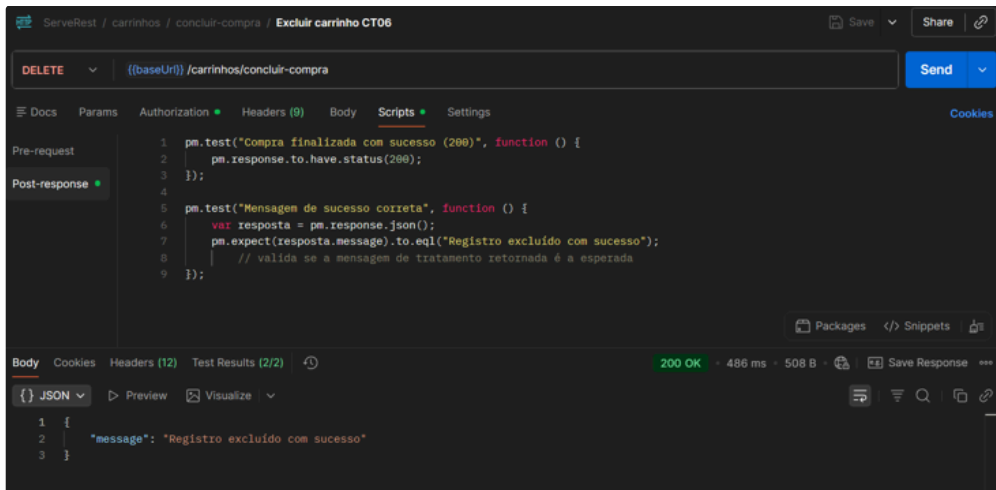
```
1 pm.test("Compra finalizada com sucesso (200)", function () {
2   pm.response.to.have.status(200);
3 });
```

```

3 });
4
5 pm.test("Mensagem de sucesso correta", function () {
6     var resposta = pm.response.json();
7     pm.expect(resposta.message).to.eql("Registro excluído com sucesso");
8 });

```

• Evidência:



2. Explicação dos Checks Automatizados (Scripts Detalhados)

A. Cenário Positivo: Validação de Login com Sucesso

- **Endpoint:** `POST /login` (com credenciais válidas).
- **Request Body (JSON):**

```

1 {
2   "email": "{{email_tour}}",
3   "password": "teste"
4 }

```

• Script:

```

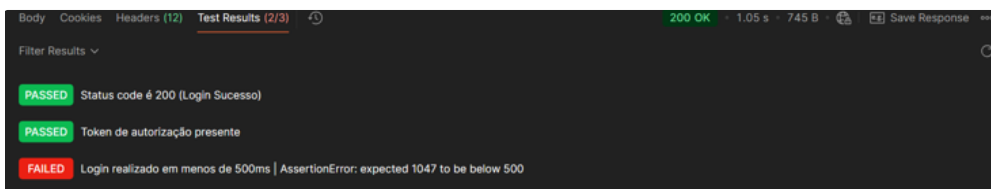
1 // Check 1: Valida Status Code
2 pm.test("Status code é 200 (Login Sucesso)", function () {
3     // postman (pm) cria teste (.test) com o nome ("") que chama function que verifica o
4     pm.response.to.have.status(200);
5     // resposta (.response) tem que ter (.to.have) status (status.200) - essa é a função
6 });
7
8 // Check 2: Valida presença do Token
9 pm.test("Token de autorização presente", function () {
10    var jsonData = pm.response.json();
11    // cria variável (var) chamada (jsonData), que recebe (.pm.response.json()) a resposta
12    pm.expect(jsonData.authorization).to.be.a('string');
13    // espera (pm.expect) que o token (jsonData.authorization) seja uma string (.to.be.a('string'))
14    pm.expect(jsonData.authorization).to.include("Bearer");
15    // espera também que o token inclua (.to.include) a palavra "Bearer" (.to.include("Bearer"))
16 });
17
18 // Check 3: Valida Performance
19 pm.test("Login realizado em menos de 500ms", function () {
20    pm.expect(pm.response.responseTime).to.be.below(500);
21    // espera que o tempo de resposta (.responseTime) da requisição seja menor que 500ms
22 });

```

• Explicação Detalhada:

- `pm.test("Status code é 200 (Login Sucesso)", function () {:`
Cria um teste do Postman, define um título para ele e chama uma função.
- `pm.response.to.have.status(200)` : Verifica se a resposta do request veio com o status 200.
- `var jsonData = pm.response.json()` : Cria uma variável chamada jsonData que recebe a resposta do request e converte em JSON.
- `pm.expect(jsonData.authorization).to.be.a('string')` : Verifica se a chave de autorização é uma string.
- `.to.include("Bearer")` : Verifica se o texto do token contém a palavra "Bearer", garantindo que é o padrão esperado.
- `pm.response.responseTime` : Pega o tempo que a requisição demorou para ir e voltar.
- `.to.be.below(500)` : Verifica se o tempo de resposta foi abaixo de 500 milissegundos.

• Resultado dos testes:



B. Cenário Negativo: Login com Senha Inválida

- **Endpoint:** `POST /login` (com senha propositalmente errada).
- **Request Body (JSON):**

```
1 {
2   "email": "{{email_tour}}",
3   "password": "SENHA_ERRADA_123"
4 }
```

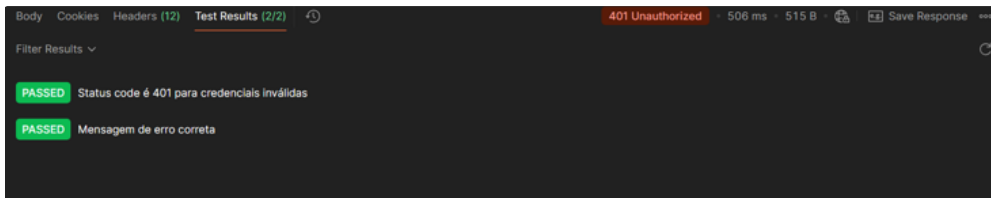
• Script:

```
◦ 1 // Check 1: Valida Status de Erro
2 pm.test("Status code é 401 para credenciais inválidas", function () {
3   pm.response.to.have.status(401);
4   // tem que ter o status de resposta 401 (unauthorized)
5 });
6
7 // Check 2: Valida Mensagem de Erro
8 pm.test("Mensagem de erro correta", function () {
9   var jsonData = pm.response.json();
10  pm.expect(jsonData.message).to.eql("Email e/ou senha inválidos");
11  // espera que o campo de mensagem (.message) da resposta seja igual (.to.eql) a "Email e/ou senha inválidos"
12 });
```

• Explicação Detalhada:

- `.to.have.status(401)` : Verifica se a resposta do request veio com o status 200. Se a API responder 200, o teste FALHA, indicando um bug grave de segurança.
- `pm.expect(jsonData.message)` : Acessa o campo de mensagem de resposta da request.
- `.to.eql("Email e/ou senha inválidos")` : Compara se o texto da mensagem é exatamente "Email e/ou senha inválidos", para garantir que o erro esteja sendo tratado corretamente.

• Resultado dos testes:



3. Registro de Bugs Encontrados

Durante a execução manual e automatizada, foram identificados os seguintes defeitos:

Bug/Melhoria #1: Campo 'preço' não aceita centavos (decimais)

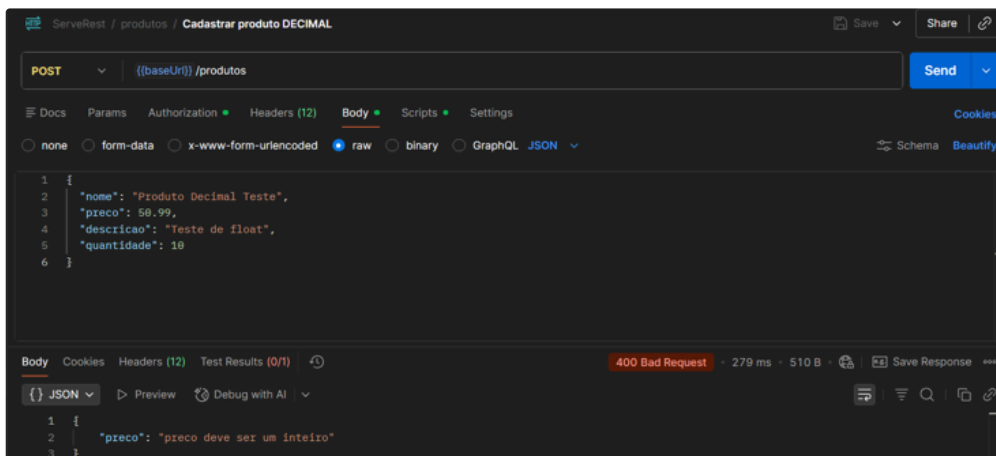
- **ID:** BUG-01
- **Título:** API retorna erro de validação ao tentar cadastrar produto com preço decimal (float)
- **Severidade:** Alta (Impede precificação correta de produtos em e-commerce)
- **Prioridade:** Alta
- **Descrição:** O endpoint de cadastro de produtos (`POST /produtos`) possui uma validação que rejeita números decimais no campo `preço` . Isso impede o cadastro de produtos com valores fracionados (ex: R\$ 9,99), o que é um requisito funcional básico para qualquer e-commerce, limitando estratégias de precificação e cálculos de descontos percentuais.
- **Endpoint:** `POST /produtos`
- **Passos para Reproduzir:**
 - ✓ Realizar login com um usuário administrador (`POST /login`) e copiar o Token gerado.
 - ✓ Configurar uma requisição `POST` para o endpoint `https://serverest.dev/produtos` .
 - ✓ No Header, incluir a chave `Authorization` com o valor do Token.
 - ✓ No Body, inserir um JSON válido, mas preencher o campo `preço` com um valor decimal (ex: `50.99`).
 - ✓ Enviar a requisição.
- **Resultado Esperado:** O sistema deveria aceitar valores do tipo `float` ou `double` (com até duas casas decimais) para representar centavos, retornando **Status 201 Created**.
- **Resultado Obtido:** O sistema retornou **Status 400 Bad Request** com a mensagem de validação "preço deve ser um inteiro".
- **Request Body:**

```
1 {
2   "nome": "Produto Decimal Teste",
3   "preco": 50.99,
4   "descricao": "Teste de float",
5   "quantidade": 10
6 }
```

• Response Body:

```
1 {
2   "preco": "preco deve ser um inteiro"
3 }
```

• Evidência:



Bug #2: API aceita e-mails com caracteres especiais e emojis

- **ID:** BUG-02
- **Título:** Falha na validação de formato de e-mail permite cadastro com caracteres inválidos (#) e emojis.
- **Severidade:** Média (Compromete a integridade dos dados e impede comunicação futura via e-mail).
- **Prioridade:** Alta
- **Descrição:** O endpoint de cadastro de usuários (`POST /usuarios`) não valida corretamente o padrão de formatação de e-mails. O sistema aceita strings contendo caracteres especiais no início (como `#`) e emojis (🚀), persistindo dados inválidos no banco de dados. Isso impede o envio de e-mails transacionais e pode causar erros em integrações futuras.
- **Endpoint:** `POST /usuarios`
- **Passos para Reproduzir:**
 - ✓ Configurar uma requisição `POST` para o endpoint `/usuarios` .
 - ✓ No Body, inserir um JSON com dados válidos de nome e senha.
 - ✓ No campo `email` , inserir um valor fora do padrão, contendo caracteres especiais e emojis (ex: `#emailinvalido🚀@qa.com`).
 - ✓ Enviar a requisição.

- **Resultado Esperado:** O sistema deveria validar o formato do e-mail e identificar caracteres inválidos ou emojis, deveria rejeitar a requisição com **Status 400 Bad Request** e uma mensagem explicativa (ex: "email deve ter um formato válido").
- **Resultado Obtido:** O sistema ignorou a formatação inválida e retornou **Status 201 Created**, criando o usuário com o e-mail corrompido.
- **Request Body:**

```
1 {
2   "nome": "Teste Email Ruim",
3   "email": "#emailinvalido🚀@qa.com",
4   "password": "teste",
5   "administrador": "true"
6 }
```

- **Response Body:**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "4WwEijzdsqu4Jm7r"
4 }
```

- **Evidência:**

